Anton Dergunov

Data Scientist · Applied Scientist · ML Engineer

London, UK · British Citizen

anton.dergunov.public360@simplelogin.com | linkedin.com/in/anton-dergunov
github.com/anton-dergunov | anton-dergunov.github.io

Professional Summary

- Over 8 years applying machine learning in production search, ranking and recommendation products — at Microsoft (Microsoft 365 Copilot, Outlook, Teams) and Yandex (web & video search), within a 20+ year engineering career; a hands-on builder of production ML systems across modelling, evaluation and deployment.
- Specialised in NLP, recommendations, retrieval and semantic search, with hands-on multimodal work (vision transformers, speech models): analysing large-scale user–AI interaction, query and conversation data to understand how people actually use AI features, and turning that into product decisions.
- Design evaluation frameworks for AI products where ''good'' is fuzzy — LLM-as-judge, synthetic data, offline and online metrics, and human-judgement tooling — and run experiments end-to-end (designed and ran dozens of online A/B experiments: roughly 30 at Microsoft and 10 at Yandex).
- Take projects end-to-end — from data and analysis through modelling, experimentation and production impact.
- Set technical direction, presented analyses and architecture proposals to product and leadership, and mentored interns, research residents and analysts.

Core Expertise

- **Focus areas**: Search ranking & relevance, recommendation systems, information retrieval & semantic search, NLP & text analytics, user–AI interaction analysis, experimentation & A/B testing.
- **Modelling**: Regression, classification, clustering, gradient boosting, deep learning, transformers, embeddings & cross-encoders, graph-based ML, fine-tuning (incl. LoRA).
- **Evaluation & analytics**: A/B design and analysis, experiment design and statistical inference, offline metric design across ranking, retrieval and classification tasks, LLM-as-judge, synthetic data, error analysis, feature/data-distribution monitoring.
- **LLMs & NLP**: LLMs, RAG, prompt design & optimization, embeddings, semantic search, query understanding, content similarity.
- **Data & infrastructure**: Python, SQL, PySpark, Azure Machine Learning, LightGBM, ONNX, Pandas, NumPy, scikit-learn, PyTorch, Weights & Biases; large-scale data pipelines and offline evaluation.
- **Programming**: Python, SQL, C++, Java, C#, Bash.

Experience

Senior Applied Scientist (Machine Learning Engineer), Microsoft — May 2018 – June 2025

Data science and ML for Microsoft 365 AI features (Copilot, Outlook, Teams). Lead data scientist for Meeting Insights recommendations and, later, the People Relevance ranking stack.

Microsoft 365 Copilot — semantic people search (NLP, retrieval, evaluation):

- Took ownership of Copilot semantic (natural-language) people search after a prior implementation improved recall but regressed precision and had its online flight halted. Classified the failing queries from real Copilot prompts, root-caused the regression to a suboptimal embedding model, presented the failure analysis and a proposed architecture to PMs and leadership, and implemented the fix — moving to a stronger existing embedding model, adding cross-encoder re-ranking, and improving how each contact's attributes are encoded for retrieval (representing a person's context as explicit, labelled fields) — improving retrieval quality over the previous model by 5.3–19.1% NDCG@10 and 5.8–15.7% Precision@10 depending on the embedding model and re-ranking (offline evaluation), paving the way for the v1 rollout. (LLMs, embeddings, cross-encoders, RAG)
- Built an offline evaluation framework from scratch: generated synthetic Copilot people-search queries grounded in a spec I authored, balanced across query classes (scaling the evaluation set to thousands of synthetic queries); computed embeddings across multiple models, retrieved contacts by vector distance, and automatically scored retrieval quality using **GPT-4o as an LLM judge** (NDCG, Precision@k), validated against a manually-labelled set; plus a side-by-side UI for manual inspection. (LLM-as-judge, synthetic data, semantic search)
- Compared embedding models and retrieval methods, including cross-encoder re-ranking and quantisation: chose a practical-size embedding model for deployment as a quality-vs-compute trade-off, and showed that even binary-quantised embeddings retained strong quality.

People Search ranking across Microsoft 365 (ranking & graph ML):

- Built from scratch the ''eyes-off'' pipeline for collecting training and test data, monitoring its quality, and training ranking and classification models with offline evaluation; optimised it to process large volumes of click-log data. The resulting models power people ranking across Microsoft products (Teams, Outlook, Bing, Windows); I deployed several to production with online gains (up to +4% click-through rate and improved time-to-success). (Azure Machine Learning, PySpark, LightGBM, ONNX)
- Built a Personalized PageRank program over organisational people graphs (from Outlook group-membership data) and used it as

ranking features, with offline gains (NDCG@1 +2.15%, NDCG@3 +1.21%); separately evaluated graph-based features and people embeddings produced by an internal effort that built large graphs from Microsoft data (people, documents, emails). Presented this work at the EMEA Science Conference.

Meeting Insights — email & document recommendations in Outlook (recommendations, NLP, weak supervision):

- Improved the relevance of the email/document recommendations behind Outlook's Meeting Insights feature, using ML and heuristics.
- Scaled the labelled training set roughly 50–100× via Azure Data Lake / U-SQL and designed the labelling heuristics, shipping improved relevance models (AUC 89%→92%, AUPRC 16%→28%); identified and removed a data bias that further lifted AUC 85%→88% and NDCG@3 0.54→0.65, making it the default production model. (PySpark, LightGBM)
- Improved content-similarity features with BERT and related architectures: verified gains across multiple datasets, ran error analysis against existing features, and achieved the first offline BERT gains for Meeting-to-Document ranking (NDCG@3 75.8→78.6). Distilled BERT into smaller models, motivated mainly by inference latency and limited GPU availability — a quality-vs-cost trade-off. (BERT, PyTorch, ONNX)
- Explored and combined data sources for training — user-interaction logs, manually-annotated data, and heuristically-labelled ''eyes-off'' data via weak supervision (Snorkel); incubated the weak-supervision approach and presented it to the Snorkel team.
- Clustered recommended documents to better understand what the system was surfacing (exploratory analysis to characterise model behaviour).
- Built data-labelling and in-product feedback tools used by around 20 people to collect evaluation data (team bug-bashes, in-product feedback), enabling more reliable model-quality estimation across projects.
- Built a model-explanation tool that decomposes model scores by feature contribution to root-cause user issues; it changed how the team worked — from guessing which feature caused a failure to pinpointing it — and motivated shipping a production explanation facility for people-search models. (model interpretability)

Understanding user–AI interactions, experimentation & communication (across the role):

- Spent roughly 20% of my time on a recurring user-issue (DSAT) triage rota — analysing real user–AI interactions (Copilot agent logs, people-search queries, Meeting Insights relevance feedback) to find failure patterns and the characteristics of successful interactions, then fixing the root cause or routing it to the right team.
- Designed and ran roughly 30 online A/B experiments end-to-end: formed hypotheses, chose primary and guardrail metrics, sized samples per stage, analysed results and translated them into decisions; for semantic search I authored the experiment spec and expected results. Monitored retention and downstream value alongside short-term engagement.
- Built self-serve feature- and data-distribution monitoring (per-scenario stats, query-type counts, significance checks) that ran inside the ML pipelines and was used widely by other engineers for error analysis and trust in the data.
- Vetted and compared candidate models offline against logged user clicks, screening them before committing to a live A/B flight.
- Communicated and influenced beyond delivery: authored specs, architecture and telemetry proposal documents read and acted on across teams; delivered internal ML lectures (clustering / DBSCAN, association-rule mining / Apriori) and Microsoft Africa Research Institute Academy talks (''Introduction to Unsupervised Learning'', ''Building Machine Learning Pipelines''); mentored 3 interns, 2 research residents, and supported analyst colleagues. (Colleague feedback: *''In depth knowledge of data science, I feel very confident in approaching Anton when I need any help on anything related to data science.''* (2021); *''I truly value your depth of knowledge, your generosity, availability and helpfulness.''* (2022); *''the project not have succeeded without your guidance and time.''* (2019).

Independent ML Research & Consulting, Self-employed — July 2025 – Present

A self-directed period building and evaluating AI/ML products, deepening hands-on depth in modern ML, and consulting.

- **Evaluation and post-training:**
  - **LLM agent evaluation test bed**: a support agent for a fictional neobank, built to iterate on LLM-agent evaluation. A deterministic-authority design keeps the model to conversation while testable code makes every money/account decision — so prompt injection has no lever to act on; the model sees only a scoped context per flow (instructions, state, that flow's tool schemas), with RAG separating public knowledge answers from gated sensitive ones. Paired with a layered evaluation stack — deterministic checks, adversarial guardrails and a calibrated LLM-as-judge over a simulated-user harness — plus experiments on judge calibration, guardrails and optimization; write-up in progress. (Python, LLMs, RAG, LLM-as-judge, guardrails, agent tool-use)
  - **Mandarin pronunciation coach**: analyses a learner's speech (speech recognition with Whisper, forced alignment with a wav2vec2 CTC model, pitch tracking) and visualises pitch and per-syllable timing against a target phrase for self-guided practice — surfacing interpretable pitch/timing feedback rather than a single score, given the genuine ambiguity of ''correct'' pronunciation. (Python, speech models, forced alignment)
- **Retrieval and ranking:**
  - **Retrieval of word and phrase usage from native speech**: builds a searchable corpus from the captions of curated YouTube channels and returns the exact video moments where a target word or phrase is spoken — timestamp-aware reconstruction of utterances from caption events, accent-tolerant surface and morphological matching of 1–5-word phrases, and deterministic ranking with cross-video diversification, served through a JSON API. Built as an experimental platform for phrase matching, speech–text alignment and example-quality ranking. (Python, information retrieval, ranking)
  - **Task-to-concept retrieval**: a retrieval test bed that maps a short task description to the most appropriate of 4,257 visual

symbols, matching on LLM-generated descriptions of each symbol rather than its name. No LLM runs on the query path, so it can serve while a planner's agenda renders, and a calibrated gate abstains when no symbol is good enough. Explores hybrid lexical-semantic retrieval, embedding models and reranking. (EMBEDDINGS, RETRIEVAL, RERANKING, CALIBRATION)

- **Agents and context engineering:**
  - **Agentic planning system**: a planning system with a deliberately designed methodology, implemented in Emacs/Org, with an LLM agent working over plans held in local files — aware of the open file and the selection, it edits notes and plans with every change reviewed as a diff; a generated context file gives it the plan's own conventions (task states, tags, saved searches), and agent skills audit the plan and route inbox items into it. (LLM AGENTS, CONTEXT ENGINEERING, EMACS LISP)
  - **Context pipeline for a coding agent**: a self-hosted pipeline that turns what I capture during the day into context an agent can file — capture from phone, browser or command line; extraction to clean Markdown from web pages, PDFs and video, using OCR and local speech transcription, with per-site handlers and no language model in the extraction path; link and title resolution; and retrieval of related notes ahead of the agent (BM25 recall, cross-encoder rerank, abstention), with the agent's token cost measured with and without it. (PYTHON, INFORMATION EXTRACTION, OCR, SPEECH-TO-TEXT, RETRIEVAL)
- **Applied LLM and speech systems:**
  - **Acervo, an LLM-written personal dictionary**: a self-hosted, offline-first vocabulary store. A pasted word or sentence becomes a structured entry — senses, definitions, glosses, examples — written by LLMs and approved by the learner, with duplicate detection; multimodal generation adds an image per sense, spoken audio, and clips of native speech selected per sense. Where a model is trusted and where it is checked was decided by experiments kept in the repository: structured output without constrained decoding, fall-through to the next model on an unusable answer, and blind comparisons for image continuity and audio encoding. (PYTHON, LLMS, IMAGE & SPEECH MODELS, EVALUATION)
  - **LexiBeat**: generates language-learning audio loops — each word spoken in the target language, then its translation, placed on a beat grid over a procedural music bed (NUMPY/SCIPY signal processing, no model), reproducible byte-for-byte from a seed. Offered as a library and a versioned HTTP service; the sound library was selected through blind A/B listening. (PYTHON, DSP, TTS)
- **From-scratch ML residency** (foundation-models-experiments; PYTORCH, GPU training, WEIGHTS & BIASES): implemented and trained from scratch — neural retrieval (dual- and cross-encoders, evaluated with the full IR metric suite) on MS MARCO; transformers and a Vision Transformer with an image→text reader (including a self-built data-collection tool for a validation set); CLIP→language-model image captioning with LoRA fine-tuning; and a complete RLHF pipeline. Deepening depth in retrieval, evaluation, transformers and LLM alignment.
- **Reinforcement learning**: attended the 2-week Deep Learning & Reinforcement Learning Summer School (Canada); studied RL via the Hugging Face Deep RL course and Sutton & Barto's *Reinforcement Learning: An Introduction*.
- **Independent ML consulting**: prototyped and compared item-response-theory and embedding-based collaborative-filtering models for a cold-start personalization problem on a large-scale real-world response dataset; selected the simplest effective model on held-out metrics (log-loss, ROC-AUC, Brier) and presented recommendations to the client — choosing the method to fit the question rather than the reverse.

Software Engineer (Machine Learning Engineer), Yandex — JANUARY 2016 – MAY 2017

Feature engineering and ML for Internet video search, at one of the world's largest search engines.

- Implemented new video features for the MatrixNet gradient-boosting ranker, improving Yandex Video Search relevance. (C++, PYTHON, MAPREDUCE)
- Built an ML pipeline for large-scale classification of online video descriptions into around 20 categories (logistic regression, LSTM, word2vec), used to improve video ranking and click-through rate; ran roughly 10 A/B experiments. (NLP, TEXT CLASSIFICATION, DEEP LEARNING)
- Collaborated across teams to improve integration of Video Search results on the main Yandex search page.

EARLIER EXPERIENCE

Lead Application Developer, JP Morgan — MAY 2017 – MAY 2018

- Built internal engineering-analytics systems monitoring engineering practices across the company, over relational data. (JAVA, PYTHON, SQL)

Senior Software Development Engineer / Research, Intel — FEBRUARY 2006 – DECEMBER 2015

Applied research and tooling for software performance analysis.

- Full-cycle development of the Intel VTune Amplifier profiler from scratch, over a SQLITE/SQL data store; owned several of its core data-visualization and data-provider components, defining their design, and mentored 5 colleagues. (C++, JAVA, PYTHON)
- Led a pattern-based (MVVM) refactor of central components, introducing proper unit tests and a test-driven approach: code duplication fell and the defect backlog dropped several-fold and stopped growing, all while feature delivery continued.
- Built a knowledge-representation system (in Python) to automatically detect performance issues in applications, for VTune Amplifier and Intel Advisor. (KNOWLEDGE REPRESENTATION)

Lecturer, Research Advisor, Lobachevsky State University — 2009 – 2012

- Developed and taught university courses in Artificial Intelligence and Databases & Expert Systems; supervised 3 bachelor and 5 master students across AI, ML, NLP and software engineering.

Earlier: software-engineering roles in the telecom domain (C++, Linux), 2003–2006.

Education

PhD in Theoretical Computer Science, Lobachevsky State University — July 2008 – July 2011
Thesis: Models, methods and tools of knowledge representation for improving performance of MPI applications.

Specialist Degree in Applied Computer Science, Lobachevsky State University — September 2001 – May 2006
Thesis: Neural network model for knowledge representation and extraction.

Writing

- Machine-learning blog — e.g. ''On Maximal Marginal Relevance'' (relevance and diversity in retrieval/ranking): anton-dergunov.github.io/blog.
- Anton Dergunov, ''Generalized algebraic data types in Haskell'', The Monad.Reader, Issue 22.